

Predictive Maintenance using LSTM for Remaining Useful Life (RUL) Estimation

Darshan Mahale, Reuven Alvares, Jai Jhunjhunwala, Vihan Jain
Veermata Jijabai Technological Institute (VJTI)
Data Structures Lab

Implementation basis: current project files in D:\projects\predictive_maintenance_project

1. Abstract

This paper presents the predictive maintenance implementation contained in the current project directory. The project estimates Remaining Useful Life (RUL) for aircraft engine degradation data using a supervised Long Short-Term Memory (LSTM) regression model. The implementation loads the FD001 subset from the predictive_maintenance_dataset directory, computes engine-wise RUL from the maximum observed cycle, applies an upper RUL clip of 125 cycles, removes selected operating and sensor columns, scales the remaining features using MinMaxScaler, and converts the data into fixed-length temporal sequences of 30 cycles. The model is a Sequential Keras network with two LSTM layers, dropout regularization, and a single ReLU-activated output neuron trained with mean squared error loss. Evaluation is performed on the FD001 test set by using the final 30-cycle sequence for each test engine and computing root mean squared error (RMSE) against the provided RUL_FD001.txt target values. In the execution performed for this paper, the project printed a final RMSE of 14.530407296633316 cycles. All methodological claims in this paper are restricted to the observed code and dataset files in the project.

2. Introduction

Predictive maintenance aims to estimate the future condition of equipment so that maintenance can be scheduled before failure. In this project, the task is formulated as Remaining Useful Life estimation, where the model predicts the number of operating cycles remaining before an engine reaches failure. The project uses multivariate time-series records from the FD001 aircraft engine degradation dataset and learns a regression model from historical run-to-failure trajectories.

The implementation is centered on temporal modeling. Each engine is represented by ordered operating cycles containing operational settings and sensor measurements. Rather than training on isolated rows, the project forms 30-cycle windows so that the model receives a short history of operating behavior before predicting the RUL associated with the next time point. This design matches the sequential nature of degradation: sensor trends across recent cycles can be more informative than a single measurement snapshot.

The scope of this paper is intentionally implementation-specific. It does not introduce external baselines, additional datasets, or methods not present in the codebase. The objective is to document the actual pipeline, model, training configuration, and evaluation output produced by the current project.

3. Related Work

Remaining Useful Life estimation is a common predictive maintenance task for systems that produce repeated condition-monitoring measurements. General approaches frame the problem as supervised regression, where historical degradation trajectories are used to learn the relationship between sensor behavior and remaining operating time. For time-series inputs, recurrent neural networks are relevant because they process ordered measurements and maintain an internal state across the sequence.

Long Short-Term Memory networks are a recurrent architecture designed to retain and update sequence information through gated operations. In the context of this project, the LSTM is used in a light and practical way: it receives a 30-cycle window of scaled sensor and operational features and outputs a single non-negative RUL prediction. This paper does not claim comparison against other techniques, because no baseline model or comparative study is implemented in the project files.

4. Dataset Description

The project uses the FD001 files located in `predictive_maintenance_dataset`: `train_FD001.txt`, `test_FD001.txt`, and `RUL_FD001.txt`. The dataset readme describes FD001 as containing 100 training trajectories and 100 test trajectories under one operating condition at sea level with one fault mode, high-pressure compressor degradation. The code loads only FD001, even though the readme also describes other FD subsets.

The data loader reads each text file as a space-separated table with no header and drops empty columns introduced by spacing. It assigns 26 columns: `id`, `cycle`, `op1`, `op2`, `op3`, and sensor columns `s1` through `s21`. The inspected training file contains 20,631 rows and 26 columns, while the inspected test file contains 13,096 rows and 26 columns. Both training and test files contain 100 unique engine identifiers. Training cycles range from 1 to 362, and test cycles range from 1 to 303.

The file `RUL_FD001.txt` provides the true remaining useful life values for the 100 test engines. During evaluation, the implementation creates one final sequence per qualifying test engine and compares the 100 predictions against these provided target values.

5. Methodology

The implemented methodology follows a direct sequence-regression pipeline. First, the training data are loaded and labeled with RUL. For each engine, the maximum cycle is computed, and the unbounded RUL at a row is the difference between that engine maximum cycle and the current cycle. The implementation then applies piecewise clipping so that any training RUL above 125 is replaced by 125.

RUL equation used in the code:

$$RUL(i,t) = \min(\max_cycle(i) - cycle(i,t), 125)$$

where `i` denotes an engine id and `t` denotes a cycle row. This equation is applied to the training dataframe before preprocessing. The target value can therefore range from 0 to 125 cycles in the processed training set.

Preprocessing removes columns that the implementation identifies as unused or unhelpful. The `cycle` column is dropped first. The columns `op3`, `s1`, `s5`, `s10`, `s16`, `s18`, and `s19` are also removed. After

excluding id and RUL from the model input, the resulting feature matrix contains 17 features. A MinMaxScaler is fit on the training feature matrix and later reused to transform test features.

Sequence generation is performed per engine. For each engine, the implementation iterates through the rows and creates an input sequence $X[i:i+30]$ for every index i in $\text{range}(\text{len}(X) - 30)$. The target for that sequence is $y[i+30]$. Thus, each training example consists of 30 consecutive cycles and the RUL at the next indexed cycle. With the inspected FD001 training file, this procedure creates 17,631 sequences with shape (17,631, 30, 17).

For the test set, the same feature columns are removed and the training scaler is applied. The implementation then selects only the last 30 scaled rows of each test engine as the prediction sequence. In the inspected FD001 test file, all 100 engines have at least 30 cycles, so 100 test sequences are produced. Predictions are compared against RUL_FD001.txt using RMSE.

RMSE equation used for evaluation:

$$RMSE = \sqrt{(1 / n) * \sum \text{from } k=1 \text{ to } n \text{ of } (y_k - \hat{y}_k)^2)}$$

where y_k is the true RUL from RUL_FD001.txt and $\hat{y}_k$ is the model prediction for the k th test engine.

Figure 1. System pipeline: FD001 text files -> load and name columns -> calculate clipped RUL -> remove selected columns -> MinMax scale features -> generate 30-cycle sequences -> train LSTM model -> predict final test-engine sequences -> compute RMSE.

6. Implementation Details

The main execution path is defined in main.py and imports load_data from src/data_loader.py, calculate_rul and preprocess_data from src/preprocessing.py, create_sequences from src/sequence.py, and build_model and train_model from src/model.py. The time_steps value is set to 30 in main.py and is used for both training sequence generation and test sequence extraction.

The neural network is implemented with tensorflow.keras.models.Sequential. It contains a first LSTM layer with 50 units and return_sequences=True, followed by Dropout(0.2). A second LSTM layer with 25 units is then followed by another Dropout(0.2). The final Dense layer has one output unit with ReLU activation, ensuring non-negative predictions. The model is compiled with the Adam optimizer and mean squared error loss.

Training is performed by model.fit with epochs=50, batch_size=32, validation_split=0.2, and an EarlyStopping callback. The callback monitors val_loss, uses patience=5, and restores the best weights. The training function also plots training loss and validation loss, adds labels and a title, and saves the figure to output/plots/loss.png.

Table 1. Model architecture configuration.

Order	Layer / Stage	Configuration	Role in implementation
1	LSTM	50 units, return_sequences=True, input_shape=(30, 17)	Processes each 30-cycle input sequence and returns a sequence representation
2	Dropout	rate=0.2	Regularizes the first recurrent representation
3	LSTM	25 units	Compresses temporal information into a fixed representation
4	Dropout	rate=0.2	Regularizes the second recurrent representation
5	Dense	1 unit, activation=relu	Outputs one non-negative RUL prediction
Compile	Training setup	optimizer=adam, loss=mse	Optimizes mean squared error during training
Fit	Training parameters	epochs=50, batch_size=32, validation_split=0.2, EarlyStopping(val_loss, patience=5, restore_best_weights=True)	Trains with validation monitoring and restores best weights

7. Results and Discussion

The implementation was executed from main.py using the project virtual environment. The run loaded the dataset, completed preprocessing, created sequences, built and trained the LSTM model, predicted on the FD001 test sequences, and printed the final evaluation metric. The printed value was:

FINAL RMSE: 14.530407296633316

This RMSE is measured in RUL cycles because the target values are remaining operating cycles. Lower RMSE indicates closer agreement between predicted RUL and the target RUL values supplied in RUL_FD001.txt. The code does not compute additional regression metrics, confidence intervals, per-engine errors, or baseline comparisons, so this paper reports only the actual metric printed by the implementation.

The training procedure also generated a loss plot. The project code saves this plot as output/plots/loss.png after training by plotting history.history["loss"] and history.history["val_loss"]. The configured maximum number of epochs is 50, with early stopping based on validation loss. In the run used for this paper, training completed after the epoch 40 log before prediction was performed.

Figure 2. Training vs validation loss: text reference to the loss curve generated by src/model.py and saved at output/plots/loss.png.

The script includes a maintenance-decision printout after RMSE calculation. This logic labels predictions below 25 cycles as Critical, below 50 cycles as Warning, and all other predictions as Healthy. These labels are threshold-based console outputs and are not used in the RMSE calculation.

8. Limitations

The implementation evaluates only the FD001 dataset. Although the dataset readme lists FD002, FD003, and FD004 descriptions, the current code paths load only `train_FD001.txt`, `test_FD001.txt`, and `RUL_FD001.txt`.

The project does not set a random seed, so retraining can produce different model weights, loss curves, and RMSE values across runs. The paper therefore reports the RMSE observed in the execution performed for this document rather than claiming a deterministic benchmark.

The code does not save the trained model, scaler, predictions, or a structured evaluation report. It only saves the training/validation loss plot and prints RMSE to the console.

The validation configuration is limited to `validation_split=0.2` inside `model.fit`. No separate validation file, cross-validation procedure, or baseline comparison is implemented.

The maintenance-decision thresholds are heuristic and not evaluated against separate maintenance labels. They should be interpreted as simple reporting logic rather than validated decision rules.

9. Future Work

Future work should begin with reproducibility and reporting improvements that fit the current implementation: setting explicit random seeds, saving the trained model and scaler, exporting predictions, and writing RMSE and training history to a structured results file.

The code could also be extended to evaluate additional FD subsets only after adding the corresponding data files and explicit loading paths. Any such extension should preserve the same discipline used in this paper: methods and results should be reported only when implemented and executed.

Additional practical improvements include replacing console emoji status labels with encoding-safe text labels, adding tests for RUL calculation and sequence generation, and validating that test-engine ordering aligns with the RUL target file.

10. Conclusion

The current project implements a complete LSTM-based RUL estimation pipeline for the FD001 predictive maintenance dataset. It loads CMAPSS-style text data, computes clipped engine-wise RUL labels, removes specified columns, scales retained features with `MinMaxScaler`, forms 30-cycle sequences, trains a two-layer LSTM regression model, and evaluates test predictions using RMSE. The execution performed for this paper printed a final RMSE of 14.530407296633316 cycles.

The implementation is concise and aligned with the core predictive maintenance objective, but its reported evidence is limited to a single FD001 run and a single RMSE metric. Within that scope, the project demonstrates the essential workflow for sequence-based RUL prediction and leaves reproducibility, broader evaluation, and reporting infrastructure as clear next steps.